Vision Transformer

Cho các mảnh ảnh trao đổi như token

Hai mảnh ảnh ở xa nhau

CNN nối vùng xa dần qua nhiều lớp convolution.

Attention có thể cho hai patch trao đổi trực tiếp.

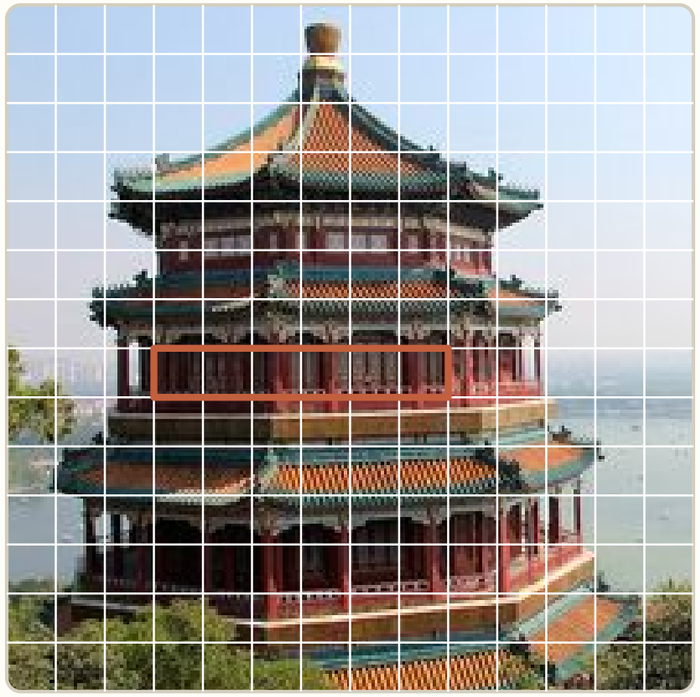
Mỗi patch cũng cần biết nó nằm ở đâu.

01

Ảnh thành patch

Ảnh thành dãy token

224×224 · patch 16 → lưới 14×14



patch 101–106 của hàng 7



196 token, mỗi token 3·16·16 = 768 số thô

Ảnh thật 224×224 chia patch 16×16 thành lưới 14×14; mỗi patch trải thành một token 768 số, đánh số theo hàng từ 0 đến 195.

Từ ảnh thành chuỗi token



chia lưới 4 × 4, mỗi patch 56 × 56 × 3

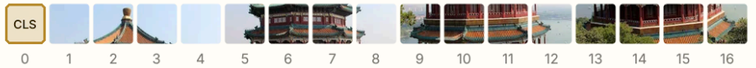


các patch tách rời, chưa trộn thông tin





hàng 3 vào vị trí 8–11 của chuỗi



ghép CLS vào đầu: chuỗi 17 token

Ảnh tháp chia lưới 4 × 4 (patch 56 × 56 cho dễ nhìn); các hàng patch trải thành chuỗi token, rồi ghép CLS ở đầu.

Đếm token

với ảnh chia hết cho .

Mỗi patch thô có số, với là số kênh.

Projection đưa mỗi patch về vector chiều.

ViT-B/16, ảnh 224: 196 token, patch thô 768 số, .

HỎI NHANH

Ảnh RGB được chia thành các patch . Có bao nhiêu patch, và mỗi patch thô dài bao nhiêu số?

A 64 patch, mỗi patch 192 số

B 16 patch, mỗi patch 192 số

C 16 patch, mỗi patch 64 số

D 4 patch, mỗi patch 768 số

Tensor chia đúng ô vuông

`images` có shape `(1, 1, 4, 4)` và `patch_size = 2`. `unfold` lấy từng ô; `permute` đưa hàng, cột patch lên trước kênh rồi mới trải patch.

```
windows = images.unfold(2, patch_size, patch_size)
windows = windows.unfold(3, patch_size, patch_size)
patches = windows.permute(0, 2, 3, 1, 4, 5).reshape(1, 4, 4)
```

Kết quả patchify

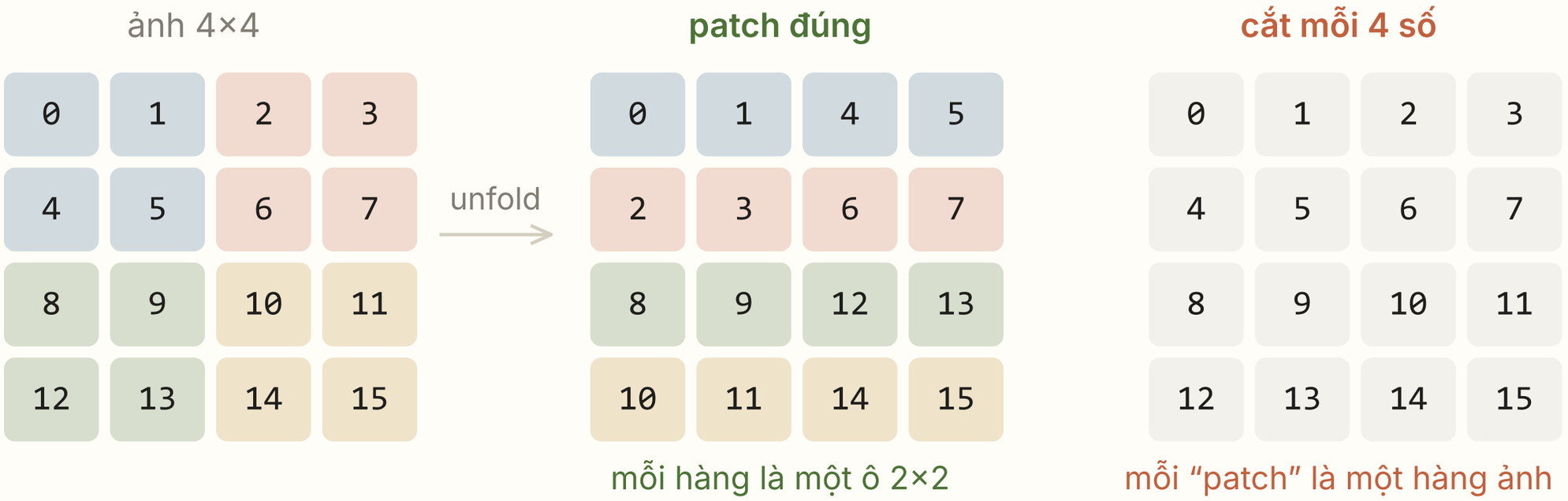
```
print(patches[0].to(torch.int64).numpy())
print(tuple(patches.shape))
```

KẾT QUẢ

```
[[ 0  1  4  5]
 [ 2  3  6  7]
 [ 8  9 12 13]
 [10 11 14 15]]
(1, 4, 4)
```

Có 4 patch, mỗi patch dài 4. Patch đầu gồm các phần tử `[0, 1, 4, 5]`, không phải bốn số liên tiếp theo hàng.

Đừng cắt dãy theo hàng



Ảnh 4×4 đánh số theo hàng (số từ code bài): unfold rồi permute cho patch đúng (0, 1, 4, 5); flatten rồi cắt mỗi 4 số chỉ lấy một hàng ảnh.

HỎI NHANH

Ảnh RGB , patch cỡ 2, nên `windows` có shape `(1, 3, 2, 2, 2, 2)`. Một bạn bỏ bước `permute` và gọi thẳng `windows.reshape(1, 4, 12)`. Hàng đầu của kết quả chứa gì?

A Ba kênh của patch trên trái, nên `permute` là thừa

B Ba kênh của bốn điểm ảnh ở hàng ảnh đầu tiên

C Kênh R của ba patch đầu tiên

D Không có gì, vì không reshape được tensor sáu chiều

Một projection dùng chung

Conv kernel , stride có thể tương đương phép gom patch.

Hai cách chỉ tương đương khi thứ tự phần tử và trọng số được xếp giống nhau.

Đối chiếu hai cách

`tokens` và `conv_tokens` là hai kết quả trên cùng một ảnh sau khi đã đặt trọng số tương ứng.

```
print(torch.allclose(tokens, conv_tokens))
```

KẾT QUẢ

```
True
```

`True`: Linear trên patch trải phẳng và Conv2d kernel 2, stride 2 cho cùng token khi trọng số xếp cùng thứ tự.

HỎI NHANH

Tăng từ 3 lên 12, giữ nguyên ảnh và patch size. Đại lượng nào không đổi?

A Số patch

B Số tham số của phép chiếu

C Chiều của mỗi vector token

D Shape trọng số của Conv2d tương đương

HỎI NHANH

Patch embedding của ViT-B/16 là `nn.Conv2d(3, 768, kernel_size=16, stride=16)`, chạy trên ảnh . Lớp này có bao nhiêu tham số?

A

B

C

D

02

Trao đổi bằng attention

Một head đọc các token

Softmax tính theo từng hàng. $\sqrt{}$ là chiều của query và key.

Class token

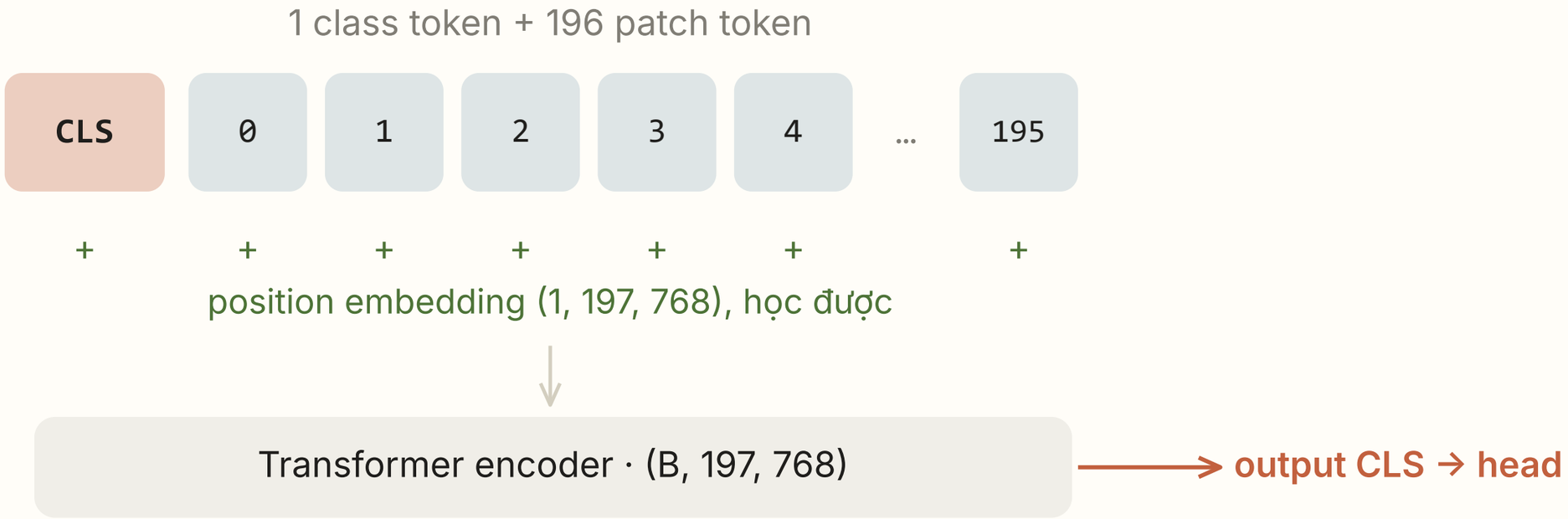
Class token là một vector học được, ghép vào đầu chuỗi, chỉ đọc các patch qua attention.

Output của nó ở lớp cuối đi vào head phân loại.

ViT-B/16, ảnh 224: 196 patch + CLS = 197 token, shape .

Mean pooling các patch là cách khác, tốt ngang khi chỉnh learning rate.

Chuỗi vào encoder



Chuỗi vào ViT-B/16: một class token ghép trước 196 patch token, cộng bảng vị trí học được, qua encoder với shape (B, 197, 768); output của CLS đi vào head.

HỎI NHANH

Lưới patch. Sau một lớp attention, patch đã đọc được patch chưa? Với conv stride 1 thì cần ít nhất mấy lớp?

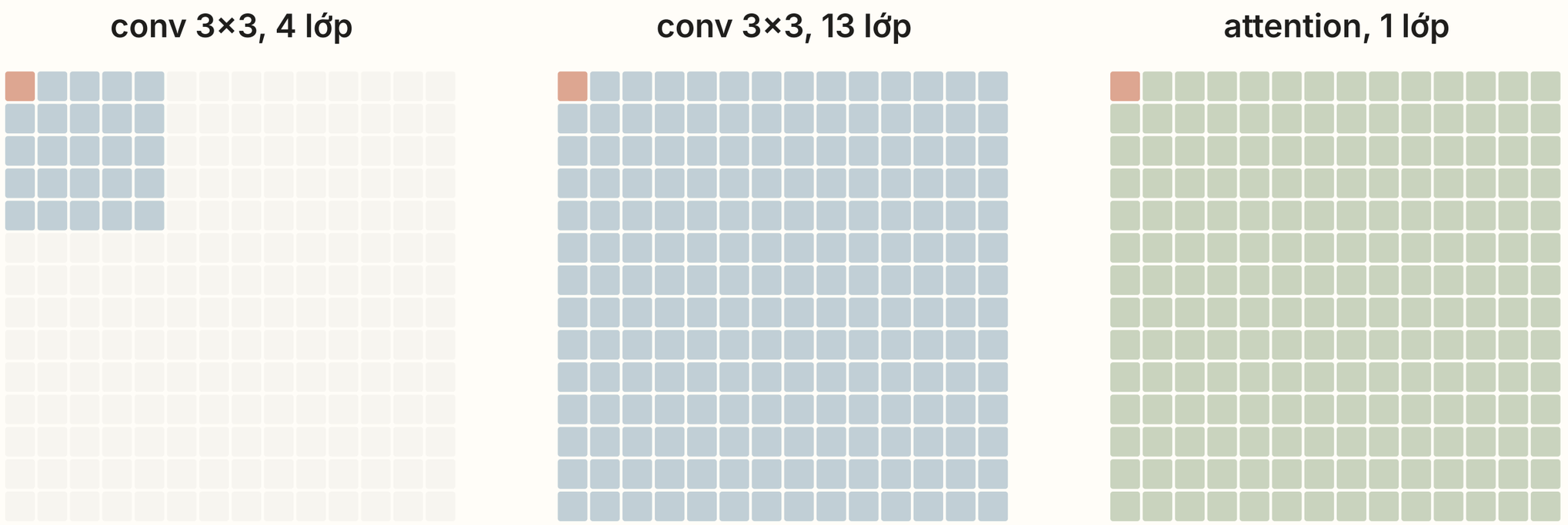
A Rồi, và cần 7 lớp conv

B Rồi, và cần 13 lớp conv

C Chưa, và cần 7 lớp conv

D Chưa, và cần 13 lớp conv

Một lớp attention, mười ba lớp conv



vùng ô (0, 0) đọc được trên lưới 14×14 patch

Từ ô góc (0, 0) của lưới 14×14: conv 3×3 stride 1 cần 13 lớp mới với tới góc đối diện; một lớp attention đã có đường đọc mọi patch.

Nội dung đổi, vị trí giữ

ảnh gốc



cùng 16 patch, đổi chỗ



không có vị trí:

mean pooling bằng nhau

cộng mã vị trí cố định:

output khác nhau

Hai ảnh có cùng tập 16 patch, chỉ khác bố cục: thiếu position embedding, self-attention rồi mean pooling cho cùng kết quả; cộng mã vị trí cố định tại chỗ thì hai ảnh tách được.

Thử hoán vị

KẾT QUẢ

```
without position: True
with fixed positions: False
```

Không có vị trí: đảo các mảnh, kết quả sau pooling không đổi.

Cộng mã vị trí cố định: cùng tập mảnh ở bố cục khác cho output khác.

Câu hỏi về cấu trúc, nên block ngẫu nhiên là đủ để kiểm.

HỎI NHANH

Ta đảo thứ tự các token ảnh, và position embedding cũng đi theo từng token. Sau đó lấy mean pooling. Vì sao kết quả có thể giữ nguyên?

A Vì position embedding chưa được train nên chưa có tác dụng

B Vì mean pooling xóa hết thông tin vị trí trong token

C Vì chỉ thứ tự danh sách thay đổi, mỗi patch vẫn giữ mã vị trí cũ

D Vì encoder ở chế độ `eval()` nên bỏ qua position embedding

Bảng vị trí của ViT gốc

Mỗi chỉ số patch và CLS có một vector học được: shape

.

Bảng 1D không ghi sẵn hàng, cột; model học ra từ dữ liệu.

Sau huấn luyện, vị trí cùng hàng hoặc cùng cột có vector giống nhau hơn.

03

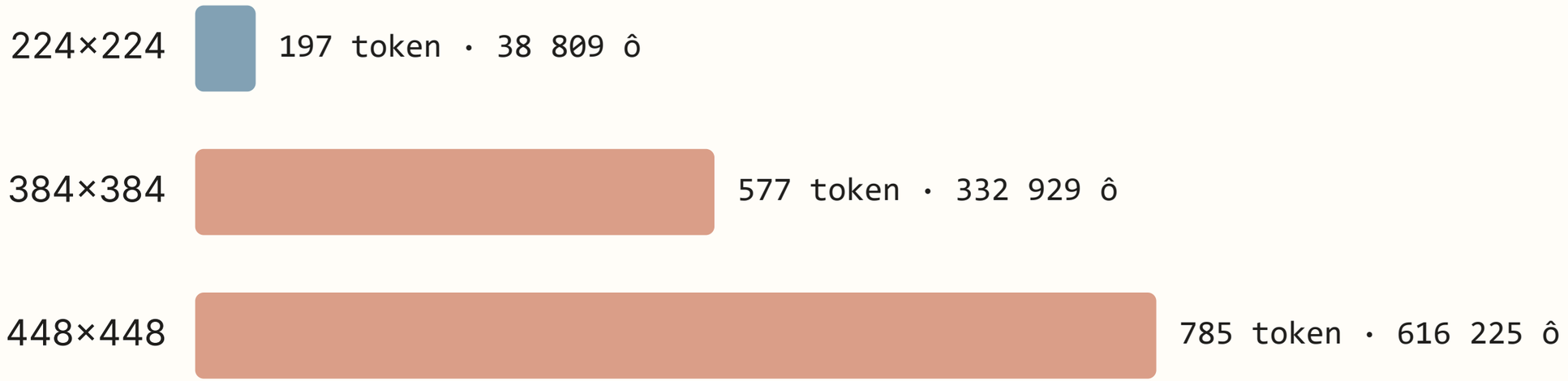
Độ phân giải, dữ liệu và ngân sách

Độ phân giải có giá

Nếu tăng gấp đôi cả và :

Giữ patch size, số patch tăng 4 lần, còn phần ma trận attention giữa các patch tăng 16 lần. Nếu thêm class token, tổng kích thước cần tính cả token đó.

Số ô attention theo cạnh ảnh



ô attention mỗi head, mỗi lớp = $(L + 1)^2$; cạnh ảnh gấp đôi thì gấp khoảng 16

Số ô của ma trận attention mỗi head, mỗi lớp của ViT patch 16: 197^2 ở ảnh 224, 577^2 ở ảnh 384, 785^2 ở ảnh 448, tức gấp khoảng 16 lần khi cạnh ảnh gấp đôi.

CNN và ViT

CNN có thiên hướng cục bộ và chia sẻ trọng số.

ViT nối vùng xa ngay lớp đầu nhưng phải học quan hệ không gian.

Bài báo ViT: chỉ train trên ImageNet (khoảng 1.3 triệu ảnh), không dùng regularization mạnh thì ViT kém ResNet cùng cỡ vài điểm.

Pretrain trên JFT-300M (303 triệu ảnh), ViT vượt.

HỎI NHANH

Train từ đầu trên 3000 ảnh. Một CNN nhỏ đạt 85% trên validation. Một ViT nhỏ có số tham số tương đương đạt 100% trên train nhưng chỉ 70% trên validation. Lý do khả dĩ nhất là gì?

A ViT thiếu giả định cục bộ nên overfit khi có ít ảnh

B ViT underfit, cần thêm lớp để tăng sức chứa

C Code ViT có lỗi, vì attention toàn cục phải tốt hơn

D Validation của ViT bị rò dữ liệu từ tập train

Đổi độ phân giải checkpoint

Tách CLS, đưa lưới về , nội suy 2D, trải lại, ghép CLS.

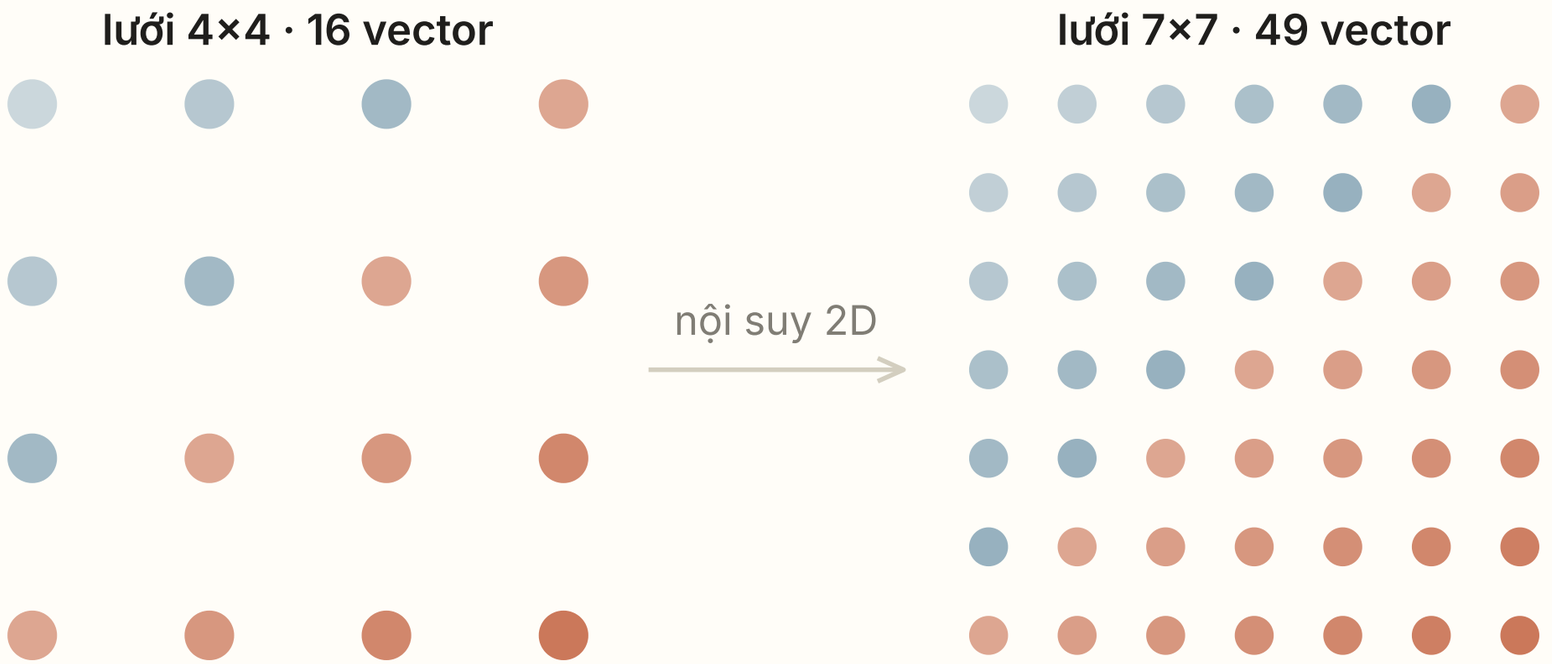
```
resized = resize_position_table(table, old_grid=4, new_grid=7)
```

KẾT QUẢ

```
(1, 17, 2) -> (1, 50, 2)
CLS: [-1.0, -1.0]
row of first column: [0.0, 0.5, 1.0, 1.5, 2.0, 2.5, 3.0]
```

Bảng tự đặt: vector mỗi patch là (hàng, cột) của nó. Sau nội suy vẫn fine-tune.

Nội suy bảng vị trí



màu theo hàng + cột của vector; CLS (−1, −1) tách riêng, giữ nguyên

Nội suy bảng vị trí từ lưới 4×4 sang 7×7 (số từ code bài): mỗi vector mới nhận tọa độ của đúng chỗ tương ứng trên lưới cũ; CLS được tách ra trước và ghép lại sau.

HỎI NHANH

Checkpoint ViT-B/16 train ở có bảng vị trí shape `(1, 197, 768)`. Bây giờ fine-tune ở , vẫn giữ patch 16. Bảng mới cần bao nhiêu vector?

A 197

B 337

C 576

D 577

Khi đi thi

- Đếm token, ước lượng trước khi tăng độ phân giải.
- Đọc cấu hình checkpoint: patch, độ phân giải, mean/std, CLS.
- In shape lưới token; kênh ở trục cuối phải `permute`.
- Chi tiết nhỏ: giữ pixel, cắt ô thay vì resize mạnh.
- So cùng validation; thời gian mỗi ảnh nhân số ảnh test phải vừa giới hạn.
- Ít dữ liệu, cấm pretrained: CNN nhỏ trước ViT.

04

Luyện đề

LUYỆN ĐỀ 1

Một đề vật lý cho mỗi mẫu một tensor shape `(64, 64, 2)`, là hai thành phần của một trường vector đo trên lưới . Một đội dùng ViT patch có class token. Chuỗi vào encoder dài bao nhiêu, và mỗi patch thô dài bao nhiêu số?

A 64 token, mỗi patch 128 số

B 65 token, mỗi patch 128 số

C 64 token, mỗi patch 192 số

D 65 token, mỗi patch 192 số

PHỎNG THEO USA-NA-AIO 2026 · Vòng 2 · Ngày 1 · Problem 2

LUYỆN ĐỀ 2

Ảnh được đưa thẳng vào `nn.Conv2d(3, 768, kernel_size=16, stride=16)`, không padding. Kết quả ra sao?

A Báo lỗi, vì 230 không chia hết cho 16

B Tự đệm thêm để ra lưới , tức 225 token

C Ra lưới , còn 6 hàng và 6 cột pixel cuối không được đọc

D Ra lưới , phần dư được gộp vào hàng và cột patch cuối

PHỎNG THEO CMU 10-601 · Exam 3 Practice · Q1.2(b)

LUYỆN ĐỀ 3

ViT đọc ảnh như một chuỗi patch. Một đội muốn giữ mọi chi tiết, nên cho mỗi pixel của ảnh thành một token. So với patch , tức 196 token, ma trận attention của mỗi head ở mỗi lớp có số ô tăng bao nhiêu lần?

A 16

B Không đổi, vì ma trận attention có kích thước

C 256

D

PHỎNG THEO NOAI 2026 Singapore · Vòng sơ loại · Câu 260

Chốt ý

ViT đi từ patch đến embedding, thêm vị trí, rồi đưa token qua Transformer.

Số token quyết định chi phí; vị trí quyết định bố cục; dữ liệu quyết định ViT có thắng CNN không.

Sau giờ học

Làm **10 câu kiểm tra cuối bài** trên trang bài học.

Chạy notebook; đổi patch size, đoán trước số token và số ô attention rồi kiểm lại.